Analysis of Parallelization Techniques on k-mer Counting for Genomic Sequences

Alex McGinn

October 18, 2026

1 Introduction

In genomic sequence analysis, genetic information is represented by long sequences containing millions or even billions of nucleotide bases, primarily adenine (A), cytosine (C), guanine (G), and thymine (T). To compare genomic samples, scientists need methods that can identify meaningful similarities even when two sequences are not exact matches. This is important because even closely related organisms or samples can contain mutations, insertions, deletions, and other sequence variations. Although genomic sequences may differ, biologically meaningful similarities can often be detected through shared local sequence patterns such as k-mers [4].

One method for identifying these local patterns is k-mer analysis. A sliding window of length k is moved across a genomic sequence, producing every contiguous substring of length k. These substrings are called k-mers, and the number of times each unique k-mer occurs can then be counted. The resulting k-mer frequencies provide a compact way to characterize and compare genomic sequences without requiring the sequences to match in their entirety. Larger values of k provide more sequence-specific information, while smaller values provide greater sensitivity to local similarities [2]. For example, if two genomic regions contain the same 15-mer, this represents a more specific local match than sharing a much shorter k-mer. However, a single base difference can disrupt several overlapping large k-mers, while shorter k-mers are less sensitive to an individual mismatch.

When analyzing a genetic sample, we do not obtain the entire genome in one clean pass. Instead, sequencing produces many shorter reads from the sample, which often overlap and contain repeated coverage of the same genomic regions. As a result, the same k-mer may appear several times across the reads. Because the complete genome may not already be known, we cannot always determine its total size directly from the sequencing output. However, as Marcais and Kingsford explain, k-mer frequency statistics can be used to estimate sequencing coverage and subsequently "derive an estimate of the genome size from (c) and the total length of the reads" [3].

Although, as k increases, the number of possible k-mers grows exponentially, leading to an increased amount of memory required to store and process k-mer counts. Alternatively, if k is shorter, we may have many matches, which would require the counter variable to have a larger range. For large genomic datasets, these computations become expensive, but there are various methods to accelerate this process using parallelization. The aim of this project is therefore to analyze the effectiveness of CPU/GPU parallelism for these types of tasks.

2 Background

To process a set of k-mer (determined by the k value; e.g. k = 4) we first move a sliding window of size k across the entire sequence and extract every k-mer substring. The first benefit is that we can now compare

genomic sequences without simply comparing their complete ordering. Another benefit is that when we break a DNA sequence into many smaller subsequences of a known size, the condensed format allows us to perform computations in parallel, leading to a needed speedup. Although, as the input dataset size and k-value increase, the ability to store the data necessary in main memory becomes increasingly difficult.

To address this problem, one strategy for storing k-mers is to index an array or a similar data structure for each possible sequence. This allows us to predict the amount of main memory necessary to store all possible combinations for a dataset and set k-mer value. For example, a dataset could have a dedicated array that contains all possible k-mer permutation for k-value set equal to five. We know whether this would be possible or not because the number of possible permutations of such a string with 4 possible bases is:

$$4^k$$

e.g. in a 5 k-mer example:

$$4^5 = 1024 \text{ possible subsequences}$$

We could then also have an array that holds the count of each k-mer in the dataset. Therefore, we know that we need 1024 representations of the four possible bases. We can then also predict the size of the array used to count those occurrences. To determine the size necessary, we then need to consider the maximum possible number of occurences a k-mer can have. This will be dictated by frequency, which we cannot outright predict. However, we can be certain of the total number of possible occurrences based on the dataset size. That is because the maximum value will always be equal to how many possible substrings there are, determined by:

$$n - k + 1$$

e.g. in a DNA Sequence of length 66,000 with k-value equal to 5:

$$66000 - 5 + 1 = 65996$$

Which means that we need array elements that go up to at least 65996. In this instance, we would know we now need elements larger than an unsigned short, which would reach capacity at 65,535

2.1 Similar Work

One similar project proposed by Marcais and Kingsford [3] introduced Jellyfish, a parallel k-mer counting algorithm designed to improve performance and memory efficiency when processing large genomic datasets. Jellyfish utilizes a multithreaded, lock-free hash table that allows concurrent updates through atomic operations rather than conventional locking mechanisms. The implementation also leverages compact data representations to reduce memory requirements. Their work demonstrates the importance of synchronization overhead and memory-efficient data structures in parallel k-mer counting, both of which will be considered when evaluating the parallelization strategies proposed in this project.

Deorowicz et al. [1] introduced KMC 2, a parallel k-mer counting algorithm that focuses on reducing memory consumption and disk I/O overhead when processing large genomic datasets. Unlike approaches that rely primarily on concurrent hash-table updates, KMC 2 partitions sequence data into smaller groups using signatures derived from minimizers. These partitions can then be processed using parallel sorting and counting operations, reducing the amount of data that must be held in memory simultaneously. The

authors also introduced (k, x)-mers to reduce redundant processing of overlapping k-mers. This approach highlights the importance of memory management, workload distribution, and I/O overhead, which are relevant considerations for the parallelization methodology proposed in this project.

3 Sequential Implementation

3.1 Dataset and FASTA Input Processing

The initial sequential implementation was developed in C++ and tested using the Escherichia coli K-12 MG1655 reference genome, obtained from the National Center for Biotechnology Information (NCBI). The reference genome contains 4,641,652 nucleotide bases and is provided in FASTA format [5].

FASTA files contain nucleotide bases separated by header lines beginning with the '>' character. The implementation reads the file line by line, concatenating nucleotide bases belonging to the same sequence and storing each completed sequence in a string.

The runtime is evaluated both before the FASTA loading operation is performed and before the sequential counting benchmark begins, allowing the execution time of the counting algorithm to be measured independently of file input operations.

3.2 Base Encoding

Each base is represented using a two-bit integer encoding, where A, C, G, and T correspond to the values 0, 1, 2, and 3, respectively. Since each base requires two bits, a k-mer of length k can be represented using $2k$ bits. The implementation then stores the encoded k-mer in an unsigned 64-bit integer using bit manipulation, which allows a maximum k-mer length of $k = 32$, although for testing purposes we have limited it to 12.

A count `vector` is allocated with 4^k elements, corresponding to every possible k-mer of length k. Each element is a `uint64_t`, resulting in a total allocation of 8×4^k bytes. As each k-mer is processed, its two-bit representation is shifted into the encoded integer. The resulting integer value is then used directly as the index into the count `vector`, allowing the corresponding k-mer count to be incremented without requiring a hash-table lookup.

Many genetic sequences will have bases that are not read correctly, which is represented in the file as an `N`. Entries that contain this character representing an invalid reading will not be used in the final count.

3.3 Sliding Window and Frequency Counting

The sequential algorithm processes each sequence using a sliding window of length k. Rather than constructing a new substring at every position, the implementation maintains an integer containing the encoded bases of the current window.

For each new base, the encoded integer is shifted left by two bits, and the encoded value of the new nucleotide is inserted using a bitwise OR operation. A bitmask is then applied to remove any bits extending beyond the current k-mer, ensuring that only the most recent k bases are retained.

Once k valid nucleotides have been processed, the resulting integer is used as an index into a frequency counting array. Each array entry stores the number of occurrences of its corresponding k-mer. This allows the frequency counts to be updated directly without requiring a hash table or additional lookup operations.

The accuracy of the output is being verified by comparison to a python script `kmerCountVerify.py`.

3.4 Time & Memory Complexity

The main components of the applications are reading the sequences from the file, initializing the counter array, and processing and incrementing each k-mer. Initializing the count of the array to default values takes $O(4^k)$ time, while processing and incrementing each k-mer only takes $O(n)$ time. Since reading from a file also takes linear time with respect to the number of bases, the overall time complexity is $O(4^k + n)$. This sequential implementation was evaluated for 3, 6, 9, and 12.

The memory constraints for this implementation require us to hold the sequence itself in main memory, as well as a `vector` of `uint_64t` that has enough memory for all 4^k possible sequences. Table 1 shows that at $k = 12$ the amount of RAM that is necessary to store all possible sequences increases dramatically.

Table 1: Sequential k-mer counting benchmark results

k	Total (ms)	Count (ms)	Std. Dev. (ms)	Peak RAM (MiB)
3	10.744	3.270	0.150	8.59
6	9.675	3.389	0.191	8.62
9	14.189	6.719	0.268	10.59
12	105.571	45.703	1.002	136.59

In Figure 1, we can see that the time it takes to process the same amount of data for different k-values also increases greatly after $k = 9$. This is likely due to the size of the count array needed for all possible values. Initializing that array to default values alone is on the order of $4^{12} = 16{,}777{,}216$. With that many possible k-mers, the larger frequency array may reduce cache efficiency and increase memory access latency, which is likely why we see such a drop off in the performance, despite the sequence size remaining constant.

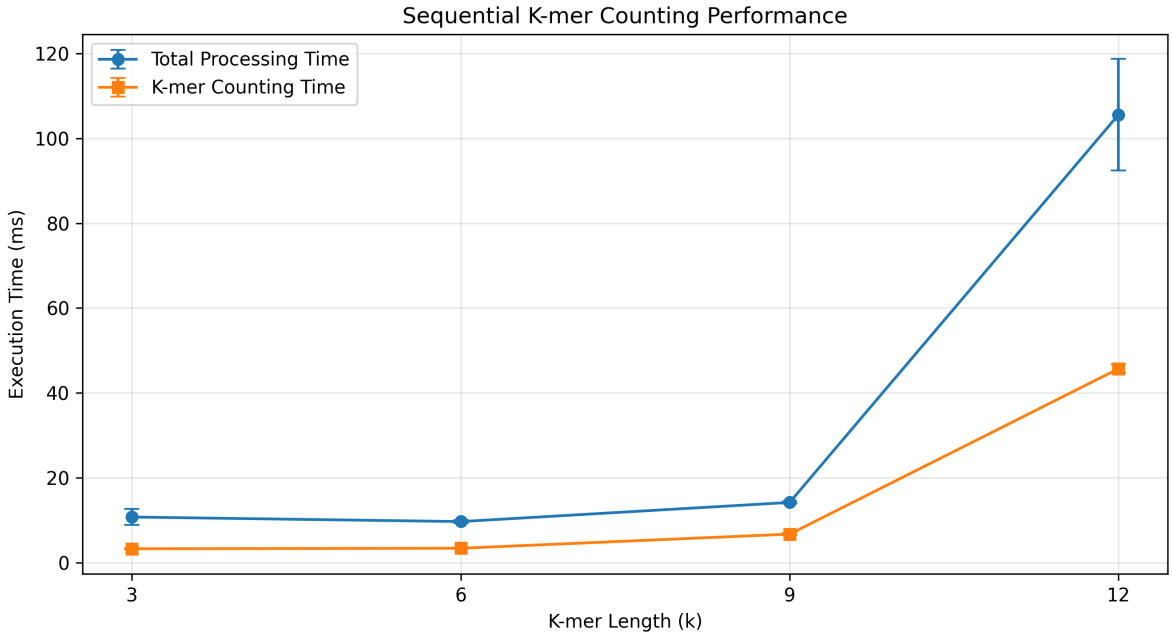


Figure 1: Sequential k-mer counting performance.

3.5 Correctness Verification

The correctness of the sequential implementation was verified against an independent Python implementation using the same E. Coli FASTA dataset. The Python implementation generates k-mers using string operations and counts their frequencies using the Counter data structure, rather than using the two-bit encoding and direct-indexed array of the C++ implementation.

Verification was performed for k-mer lengths of 3, 6, 9, and 12. For each configuration, the frequency of every unique k-mer was compared between the two implementations. All four configurations produced identical frequency distributions with zero mismatches, confirming the correctness of the sequential implementation for the tested dataset and k values.

4 Parallelization Methodology

The objective of this project is to analyze the effects of parallelization on k-mer counting for genomic sequences. The algorithm will then be written using OpenMP for parallelization on multiple CPU cores. Another implementation will utilize CUDA for GPU parallelization.

Experiments will vary the dataset size and k-mer length to evaluate the performance of both OpenMP and CUDA implementations. Both approaches will be compared against the sequential baseline in terms of execution time, speedup, and memory consumption. For OpenMP, the evaluation will examine the effects of thread count, scheduling, synchronization overhead, and the cost of combining thread-local frequency arrays. For CUDA, both GPU kernel execution time and total execution time will be measured, including host-to-device and device-to-host memory transfers. The results will be used to compare the scalability of each approach and determine under which conditions parallelization provides the greatest performance improvement.

4.1 OpenMP CPU Parallelization

The initial implementation will use OpenMP to distribute the k-mer counting workload across multiple CPU threads. The input sequences will be partitioned into smaller regions and then distributed across the threads to allow processing of different portions of the data. However, k-mers are generated using overlapping windows, so we must account for the necessary overlap in between partitions.

There are three approaches that can be taken when working with OpenMP and our current design. First, we could use the same count array and share it between the threads, but this requires some sort of protection to ensure thread safety. Atomics could be a good solution, and requires minimal changes to the sequential logic. Although, if many of the threads are attempting to increment the same counter, contention could negatively affect the performance impact.

Another method would be to assign each thread with its own array, and then consolidate the data after each thread finishes counting. This would likely lead to an increased problem with the amount of memory required for larger k, which was encountered in the sequential approach. This is because we would have to initialize a 4^k array for every thread. For 8 threads and $k = 12$ this would already take up 1GB of space.

The last method would be partitioned ownership, in which each thread is designated owner of some part of the shared array. The data is distributed, but when a thread encounters a k-mer that does not belong to its partition, it instead determines the owner and sends the k-mer to their buffer.

The first two methods are simpler to implement, and provide useful comparisons between synchronization and memory overhead. The partitioned ownership is more complex, but would provide an additional

comparison by replacing conflicting shared updates with communication overhead. An initial implementation using local counter arrays is planned, and then a more detailed attempt into the partitioned ownership method.

4.2 CUDA GPU Parallelization

A second parallel implementation will use CUDA to perform k-mer counting on the GPU. Sequence data will be transferred from RAM into the device memory of the GPU. The initial implementation will assign ranges of k-mer starting positions to GPU threads. Each thread will encode its assigned k-mers using the same two bit representation as the sequential implementation.

The encoded k-mers resulting from this task will be used to index a frequency array stored in GPU global memory. This implementation will also have to deal with race conditions or memory corruption. Atomic increments should also work for this implementation, similar to how they are planned for OpenMP.

The CUDA implementation will be evaluated by measuring the execution time of the counting kernel separately from the time required to transfer sequence data between the host and device memory. The initialization of the GPU count array and the transfer of the resulting frequencies back to host memory will also be included when measuring total processing time. The effects of different k-mer lengths, dataset sizes, and GPU thread configurations will be evaluated to determine how these factors influence performance and memory utilization.

4.3 CPU & Compiler Info

The sequential implementation was compiled using the GNU C++ compiler (g++) with the C++17 language standard and the -O2 optimization flag. This optimization level enables compiler optimizations intended to improve execution performance while retaining the same algorithmic behavior. The implementation was executed under AlmaLinux 9.6 within WSL2 on a Windows host. Each benchmark configuration was executed ten times to measure average execution time and the variation between trials.

- **CPU:** AMD Ryzen 7 7700 8-Core Processor
- **GCC:** g++ (GCC) 11.5.0 20240719 (Red Hat 11.5.0-14)
- **Architecture:** x86_64
- **Physical Cores:** 8
- **Hardware Threads:** 16 (2 per core)
- **L1 Data Cache:** 256 KiB total, 32 KiB per core
- **L1 Instruction Cache:** 256 KiB total, 32 KiB per core
- **L2 Cache:** 8 MiB total, 1 MiB per core
- **L3 Cache:** 32 MiB
- **Execution Environment:** WSL2 / Microsoft Hyper-V

5 Execution

```
make
./scripts/benchmark.sh
./out <k-val> <FASTA_file> [output_counts.csv]

# Correctness Check
./scripts/verify_counts.sh
```

References

[1] Sebastian Deorowicz, Marek Kokot, Szymon Grabowski, and Agnieszka Debudaj-Grabysz. Kmc 2: fast and resource-frugal k-mer counting. *Bioinformatics*, 31(10):1569–1576, 2015.

[2] John A. Lees, Minna Vehkala, Niko Välimäki, et al. Sequence element enrichment analysis to determine the genetic basis of bacterial phenotypes. *Nature Communications*, 7:12797, 2016.

[3] Guillaume Marcais and Carl Kingsford. A fast, lock-free approach for efficient parallel counting of occurrences of k-mers. *Bioinformatics*, 27(6):764–770, 2011.

[4] Camille Moeckel, Manvita Mareboina, Maxwell A. Konnaris, et al. A survey of k-mer methods and applications in bioinformatics. *Computational and Structural Biotechnology Journal*, 23:2289–2303, 2024.

[5] National Center for Biotechnology Information. Escherichia coli str. k-12 substr. mg1655, complete genome. `https://www.ncbi.nlm.nih.gov/nuccore/NC_000913.3`, 2013. NCBI RefSeq accession NC_000913.3; Assembly GCF_000005845.2; Accessed: October 17, 2026.